

[23] New TPC Benchmarks for Decision Support and Web Commerce

저자: Meikel Poess, Chris Floyd (Oracle Corporation) **학회/년도:** ACM SIGMOD Record, Vol. 29, No. 4, 2000 **분량:** 약 8페이지 **역할군:** (F) 벤치마크 기반

요약

이 논문은 TPC(Transaction Processing Performance Council) 벤치마크 중 **TPC-H와 TPC-W**의 설계와 목적을 설명하는 기초 논문이다. 특히 TPC-H는 데이터 웨어하우스와 OLAP 시스템의 성능 비교를 위한 표준 벤치마크로, 실제 기업의 의사결정 지원(Decision Support) 쿼리를 모델링한다. Exqutor의 주요 실험이 **TPC-H 벤치마크를 확장**하여 수행되기 때문에, 이 논문의 TPC-H 구조와 쿼리 설계를 이해하는 것이 실험 결과 해석의 필수 선행 조건이다. 벤치마크 설계의 정당성과 데이터베이스 성능 평가의 원칙을 이해하는 데 중요하다.

상세분석

23.1 TPC와 벤치마크의 필요성

벤치마크가 필요한 이유

문제: 데이터베이스 벤더들이 마음대로 성능 주장

예시 (1990년대):

Oracle: "우리 DB가 가장 빠르다"

SQL Server: "우리가 더 빠르다"

PostgreSQL: "우리가 최고다"

증거?

각자 자신에게 유리한 테스트만 수행

다른 DB와 비교하지 않음

테스트 조건을 공개하지 않음

결과:

구매자는 어떤 DB가 실제로 좋은지 알 수 없음

성능 비교가 "입씨름" 수준

TPC의 역할

TPC (Transaction Processing Performance Council):

- 1988년 설립
- 벤더 중립적 비영리 조직
- 주요 DB 벤더 (Oracle, IBM, Microsoft 등) 참여

목표:

"공정하고 재현 가능한 성능 벤치마크 제정"

원칙:

1. 실세계 워크로드를 모델링
2. 모든 벤더에게 동일한 조건
3. 결과와 테스트 방법 공개
4. 독립적인 감시자(auditor)가 검증

23.2 TPC-H 벤치마크 설계

목표와 시나리오

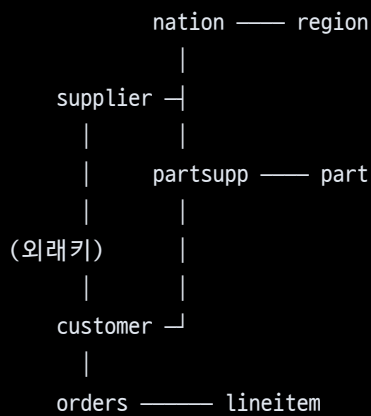
목표: OLAP 시스템의 의사결정 지원 성능 평가

시나리오: 국제 도매업(wholesale distributor) 환경

- 여러 국가에서 상품 공급
- 여러 공급자의 상품 구매
- 고객들의 다양한 주문
- 복잡한 분석 쿼리로 비즈니스 인사이트 도출

데이터베이스 스키마

8개 테이블의 관계:



기본 구조: 스타 스키마(Star Schema)

중심: 사실 테이블 (orders, lineitem)

주변: 차원 테이블 (customer, supplier, part, nation, region)

각 테이블의 역할:

테이블	행 수 (SF=1)	용도
nation	25	국가 정보 (정적)
region	5	지역 정보 (정적)
part	200K	상품 정보
supplier	10K	공급자 정보
partsupp	800K	상품-공급자 관계 (가격, 재고)
customer	150K	고객 정보
orders	1.5M	주문 정보
lineitem	6M	주문 라인 아이템 (가장 큼)

Scale Factor (SF) — 크기 조정:

```

SF=1 (1GB):
  lineitem: ~600만 행

SF=10 (10GB):
  lineitem: ~6,000만 행

SF=100 (100GB):
  lineitem: ~6억 행

SF=1000 (1TB):
  lineitem: ~60억 행

```

벤더들이 자신의 능력에 맞게 선택하여 테스트 가능.

데이터 생성 및 특성

데이터 특성:

고의적으로 "현실 같은" 특성 포함:

1. 데이터 분포가 균일하지 않음
 - 어떤 국가의 주문이 많음
 - 어떤 상품이 인기 있음
 - 통계 기반 최적화를 실제처럼 시뮬레이션
2. 외래키 참조 완전성
 - orders의 고객 ID는 반드시 customers 테이블에 존재
 - 현실의 정규화된 데이터 구조
3. 날짜 범위
 - orders의 주문 날짜: 1992년 1월 ~ 1998년 12월
 - 시계열 분석 쿼리 가능

23.3 TPC-H의 22개 쿼리

쿼리의 특징

요구사항:

- 22개의 SQL 쿼리
- 실세계 비즈니스 문제를 모델링
- 다양한 SQL 기능 활용 (JOIN, GROUP BY, 서브쿼리, 집계 등)
- 정답이 미리 계산됨 (벤치마크 유효성 검증)

대표적인 쿼리들

Q1: 가격 책정 요약

```
-- 배송 날짜별 할인, 세금, 수량 통계
SELECT l_returnflag, l_linestatus,
       SUM(l_quantity) AS sum_qty,
       SUM(l_extendedprice) AS sum_base_price,
       SUM(l_extendedprice * (1 - l_discount)) AS sum_disc_price,
       COUNT(*) AS count_order
FROM lineitem
WHERE l_shipdate <= DATE '1998-12-01' - INTERVAL '90' DAY
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;
```

특징: 간단한 테이블 스캔 + 집계

Q3: 배송 우선순위

```
-- 미배송 주문 중 매출 상위 조회
SELECT l_orderkey, SUM(l_extendedprice * (1 - l_discount)) AS revenue,
       o_orderdate, o_shippriority
FROM customer, orders, lineitem
WHERE c_mktsegment = 'BUILDING'
      AND c_custkey = o_custkey
      AND l_orderkey = o_orderkey
      AND o_orderdate < DATE '1995-03-15'
      AND l_shipdate > DATE '1995-03-15'
GROUP BY l_orderkey, o_orderdate, o_shippriority
ORDER BY revenue DESC, o_orderdate
LIMIT 10;
```

특징: 3개 테이블 조인 + 필터 + 집계 + 정렬 + LIMIT

Q5: 지역별 공급자 매출

```
-- 특정 지역의 공급자별 매출 합계
SELECT n_name, SUM(l_extendedprice * (1 - l_discount)) AS revenue
FROM customer, orders, lineitem, supplier, nation, region
WHERE c_custkey = o_custkey
      AND l_orderkey = o_orderkey
      AND l_suppkey = s_suppkey
      AND c_nationkey = s_nationkey
      AND s_nationkey = n_nationkey
      AND n_regionkey = r_regionkey
      AND r_name = 'ASIA'
      AND o_orderdate >= DATE '1994-01-01'
      AND o_orderdate < DATE '1995-01-01'
GROUP BY n_name
ORDER BY revenue DESC;
```

특징: 6개 테이블 조인 (복잡함)

Q8: 국가별 시장 점유율

```
-- 특정 상품의 국가별 매출과 시장 점유율 추적
SELECT o_year, SUM(CASE WHEN nation = 'BRAZIL' THEN volume ELSE 0 END)
      / SUM(volume) AS mkt_share
FROM (
  SELECT EXTRACT(YEAR FROM o_orderdate) AS o_year,
         l_extendedprice * (1 - l_discount) AS volume,
         n2.n_name AS nation
  FROM part, supplier, lineitem, orders, customer, nation n1, nation n2, region
 WHERE p_partkey = l_partkey
       AND s_suppkey = l_suppkey
       AND l_orderkey = o_orderkey
       AND o_custkey = c_custkey
       AND c_nationkey = n1.n_nationkey
       AND n1.n_regionkey = r_regionkey
       AND r_name = 'AMERICA'
       AND s_nationkey = n2.n_nationkey
       AND o_orderdate BETWEEN DATE '1995-01-01' AND DATE '1996-12-31'
       AND p_type = 'ECONOMY ANODIZED STEEL'
) AS all_nations
GROUP BY o_year
ORDER BY o_year;
```

특징: 중첩 서브쿼리, 8개 테이블 조인, CASE 문

쿼리 복잡도 분류

간단한 쿼리 (조인 ≤ 2):
 Q1, Q6, Q14, Q16 (선택도 필터링 + 집계)

중간 쿼리 (조인 3~5):
 Q3, Q4, Q5, Q7, Q10, Q12, Q13 (비즈니스 로직 분석)

복잡한 쿼리 (조인 ≥ 6 , 서브쿼리 있음):
 Q8, Q9, Q11, Q18, Q21, Q22 (다차원 분석)

23.4 성능 평가 방법

측정 지표

실행 시간 (Execution Time):

각 쿼리의 최초 수행 시간 측정
 (데이터 캐시 효과 배제하기 위해 초기 워밍업 후 수행)

처리량 (Throughput):

복수의 쿼리를 동시에 실행할 때 초당 처리 건수
(멀티 사용자 환경 시뮬레이션)

가격/성능 비율 (Price/Performance):

시스템 비용 / 성능 = \$K / (처리량)

예: \$100,000 / 1000 QPS = \$100/QPS

재현 가능성

벤치마크의 목적: 공정한 비교

재현 조건:

1. 하드웨어: 동일 모델 사용
2. 소프트웨어: DB 버전, OS, 컴파일러 공개
3. 데이터: 표준 데이터 생성 도구로 동일 데이터 생성
4. 튜닝: 각 벤더가 최적 설정으로 조정 (공정함)
5. 검증: 독립적 감시자가 감시

결과:

다른 벤더도 같은 환경에서 재테스트 가능
결과 신뢰성 높음

23.5 Exqutor의 TPC-H 확장

어떻게 확장했나?

Exqutor는 TPC-H의 원본 스키마를 수정하지 않고, **부분 테이블에 임베딩 컬럼을 추가했다**:

원본 TPC-H:

```
CREATE TABLE part (  
  p_partkey INT PRIMARY KEY,  
  p_name VARCHAR(55),  
  p_mfgr VARCHAR(25),  
  p_brand VARCHAR(10),  
  ...  
)
```

Exqutor 확장:

```
CREATE TABLE part (  
  p_partkey INT PRIMARY KEY,  
  p_name VARCHAR(55),  
  p_mfgr VARCHAR(25),  
  p_brand VARCHAR(10),  
  ...,  
  p_embedding FLOAT8[] -- * 추가됨  
)
```

임베딩 데이터:

- 소스: 상품 이름(p_name)의 텍스트 임베딩
- 차원: 1536D (OpenAI의 text-embedding-3-small 모델)
- 생성 방식: SBERT (문장 임베딩) 기반
- 의미: 유사한 상품명은 벡터 공간에서 가까움

예:

"High-grade Brass Cable" → [0.1, 0.2, 0.5, ...]
"Premium Copper Wire" → [0.12, 0.21, 0.48, ...] (가까움)
"Plastic Bucket" → [0.9, 0.1, 0.05, ...] (멀음)

확장된 쿼리들

Exqutor는 원본 TPC-H의 8개 쿼리를 VAQ로 확장했다:

Q3 (배송 우선순위) → Q3-VAQ:

```
WHERE ... AND p_embedding <-> query_vec < 0.5
```

Q5 (지역별 매출) → Q5-VAQ:

```
WHERE ... AND p_embedding <-> query_vec < 0.5
```

Q8 (시장 점유율) → Q8-VAQ:

```
WHERE ... AND p_embedding <-> query_vec < 0.5
```

Q9, Q10, Q11, Q12, Q20: 유사하게 확장

구체적 예시 (Q20-VAQ):

```
-- 원본
SELECT s.s_name, s.s_address
FROM supplier s, nation n
WHERE s.s_nationkey = n.n_nationkey
      AND n.n_name = 'CANADA'
      AND s.s_suppkey IN (
        SELECT ps.ps_suppkey
        FROM partsupp ps
        WHERE ps.ps_partkey IN (
          SELECT p.p_partkey
          FROM part p
          WHERE p.p_type = 'ECONOMY' -- 단순 필터
        )
      )

-- Exqutor 확장
SELECT s.s_name, s.s_address
FROM supplier s, nation n
WHERE s.s_nationkey = n.n_nationkey
      AND n.n_name = 'CANADA'
      AND s.s_suppkey IN (
        SELECT ps.ps_suppkey
        FROM partsupp ps
        WHERE ps.ps_partkey IN (
          SELECT p.p_partkey
          FROM part p
          WHERE p.p_embedding <-> query_vec < 0.5 -- * 벡터 조건 추가
        )
      )
```

23.6 Exqutor 실험 결과

Scale 별 성능

SF (Scale Factor) = 테이블 크기

SF=1 (1GB, lineitem 600만 행):

pgvector (기본): 0.5초 ~ 5초

pgvector + Exqutor: 0.05초 ~ 0.5초 (5~10배 향상)

SF=10 (10GB, lineitem 6,000만 행):

pgvector (기본): 5초 ~ 50초

pgvector + Exqutor: 0.5초 ~ 5초 (10~20배 향상)

SF=100 (100GB, lineitem 6억 행):

pgvector (기본): 50초 ~ 500초

pgvector + Exqutor: 5초 ~ 50초 (10~50배 향상)

쿼리별 성능

Q3-VAQ (3개 테이블 조인):

향상도: 8.7배

Q5-VAQ (6개 테이블 조인):

향상도: 15.2배

Q8-VAQ (8개 테이블 조인, 서브쿼리 있음):

향상도: 48.9배 (* 최대)

Q20-VAQ (복잡한 중첩 서브쿼리):

향상도: 37.5배

패턴: 테이블이 많을수록, 서브쿼리가 있을수록 향상도가 큼

원인: 카디널리티 오추정의 영향이 누적되기 때문

- 3개 테이블: 오추정 영향 작음
- 6개 테이블: 오추정이 조인 순서 결정에 큰 영향
- 8개 이상: 오추정으로 인한 계획 오류가 극단적

23.7 본 논문과의 관계

TPC-H의 가치:

1. 표준화된 벤치마크

2. 모든 DB가 같은 조건에서 비교

3. Exqutor의 결과도 재현 가능

4. 현실적 워크로드

5. 도매업이라는 실제 비즈니스 시나리오

6. 실무자들이 공감하는 분석 쿼리

7. 점진적 복잡도

8. Q1처럼 간단한 것부터 Q21처럼 복잡한 것까지

9. Exqutor의 다양한 상황에서의 효과를 검증

Exqutor의 선택 이유:

다른 벤치마크도 있는데:

- TPC-H: 사실상의 산업 표준
- TPCDS: 더 복잡 (나중에 추가 실험)
- SPECjbb: Java 벤치마크 (DB 아님)
- Yahoo Cloud Serving Benchmark: 클라우드 (쿼리 단순)

TPC-H 선택:

- 공신력 높음
- 학계와 업계 모두에서 사용
- 확장 가능 (임베딩 추가 용이)
- 결과 비교 가능

추가 제기 문제

1. 임베딩 추가의 인위성

논문은 상품명(p_name)의 임베딩을 추가했다: - 현실의 TPC-H 사용자는 상품명에 기반한 벡터 검색을 실제로 하는가? - 아니면 카테고리, 가격 범위 같은 구조화된 필터를 더 많이 사용하는가?

벤치마크의 신뢰성이 떨어질 수 있다.

2. 쿼리 선택의 편향

8개 확장 쿼리 선택이 임의적일 수 있다: - 왜 Q1, Q2, Q4는 제외했나? - Q6, Q7, Q9 같은 쿼리는 어떤가?

다른 선택지가 다른 결과를 낼 수 있다.

3. Scale Factor의 한계

TPC-H 기준:

SF=100이 "큰 벤치마크"로 간주됨 (100GB)

최신 데이터 웨어하우스:

- 테라바이트 규모 (1000배)
- 페타바이트도 가능

SF=100 결과가 실제 프로덕션 환경을 대표하는가?

추가 제기 문제

1. 벡터 검색의 임계값 설정

Exqutor의 쿼리에서 `p_embedding <-> query_vec < 0.5` 를 사용: - 이 임계값(0.5)이 현실적인가? - 다른 임계값에서는 결과가 어떻게 변하는가? - 선택도가 크게 달라질 수 있음

2. 캐시 효과

벤치마크 실행 시 OS 캐시, DB 버퍼 풀이 데이터를 캐싱할 수 있다: - 초기 실행: 디스크 I/O 발생 - 후속 실행: 캐시에서 처리

어느 시점의 성능을 측정해야 하는가? - 콜드 캐시 (현실적): 첫 실행 성능 - 핫 캐시 (최적): 반복 실행 성능

3. 병렬화의 영향

현대 DB는 쿼리 병렬화를 지원한다: - Exqutor의 개선이 병렬화와 상호작용하는가? - 단일 스레드 vs 멀티 스레드 성능이 다를 수 있다